

- **Part 1: Foundations of Node.js** – Initializing your runtime environment, installing packages, creating your first `index.html` file, and building a primitive static file server using native `http` and `fs` modules.
- **Part 2: Upgrading to Express.js** – Streamlining route handling, setting up a static file asset distribution architecture (`public/`), and introducing automated workflow watching tools (`nodemon`).
- **Part 3: User Authentication Logic** – Cryptographically hashing secrets on the backend with `bcryptjs`, setting up user request validation paths, and utilizing stateful session cookies (`express-session`) to shield secure `/dashboard` endpoints.
- **Part 4: Scaling Up with MongoDB Atlas & Mongoose** – Isolating access keys into hidden local system matrix modules (`.env`), creating document structural constraints with schemas, and pushing data persistently to a production database.
- **Part 5: Production Deployment Workflows** – Preparing environment variables, managing branch builds with GitHub, configuring cloud platforms (such as Render), and exposing the app to the internet with an SSL-secured URL.

Now that your web application is live on the internet and securely storing data in a cloud database, we are ready to move into a highly critical phase of backend engineering: **RESTful APIs, JSON communication, and Database Relationships**.

Right now, when your forms send data, your server responds by sending back a whole new webpage or raw text blocks. In modern web development, backend servers act as **APIs** (Application Programming Interfaces). They don't send back design layout files; instead, they send and receive raw data structured as **JSON** (JavaScript Object Notation). This lets front-end frameworks (like React or vanilla JavaScript) fetch data and update the screen instantly without needing a full page reload.

Part 5: Transforming Your Application into a RESTful API

We will add a feature where logged-in users can save private "Notes" to their accounts. This will teach you how to manage data relationships in MongoDB (linking a note to a specific user) and how to handle API routes.

Step 5.1: Create the Note Data Model

We need to create a structural model for a note and explicitly link it to a user's record using a MongoDB reference (`ObjectId`).

Open your `app.js` file, look at where you defined your `User` schema, and add this code right beneath it:

```
// --- DEFINE NOTE SCHEMA & MODEL ---
const noteSchema = new mongoose.Schema({
  title: { type: String, required: true },
  content: { type: String, required: true },
  // This establishes a database relationship linking this note to a User ID
  userId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  createdAt: { type: Date, default: Date.now }
});
const Note = mongoose.model('Note', noteSchema);
```

Step 5.2: Configure Express for JSON Operations

By default, Express reads form inputs via URL-encoded payloads. To make it read raw JSON objects sent via modern frontend applications, we need to add one line of middleware.

Add this near your other middleware declarations in `app.js`:

```
// Middleware to parse incoming JSON payloads
app.use(express.json());
```

Step 5.3: Build the API Endpoints (CRUD)

CRUD stands for Create, Read, Update, and Delete. These are the four core actions you perform on a database. Let's create secure API endpoints that only allow logged-in users to interact with their own notes.

Append these REST routes to your `app.js` file before your `app.listen` block:

```
// 1. CREATE: Add a new private note
app.post('/api/notes', async (req, res) => {
```

```

if (!req.session.user) {
  return res.status(401).json({ error: 'Unauthorized. Please log in.' });
}

try {
  const { title, content } = req.body;
  // Find the current logged-in user's database object
  const currentUser = await User.findOne({ username: req.session.user });

  const newNote = new Note({
    title,
    content,
    userId: currentUser._id // Link the note to this user's unique ID
  });

  await newNote.save();
  res.status(201).json({ message: 'Note saved successfully!', note: newNote });
} catch (err) {
  res.status(500).json({ error: 'Failed to create note.' });
}
});

// 2. READ: Fetch all notes belonging exclusively to the logged-in user
app.get('/api/notes', async (req, res) => {
  if (!req.session.user) {
    return res.status(401).json({ error: 'Unauthorized' });
  }

  try {
    const currentUser = await User.findOne({ username: req.session.user });
    // Only pull notes matching this specific user's ID
    const userNotes = await Note.find({ userId: currentUser._id }).sort({ createdAt: -1 });

    res.status(200).json(userNotes);
  } catch (err) {
    res.status(500).json({ error: 'Failed to retrieve notes.' });
  }
});

// 3. DELETE: Remove a note safely by its unique ID
app.delete('/api/notes/:id', async (req, res) => {
  if (!req.session.user) {
    return res.status(401).json({ error: 'Unauthorized' });
  }

  try {
    const currentUser = await User.findOne({ username: req.session.user });

    // Ensure the note exists AND actually belongs to the user trying to delete it
    const noteToDelete = await Note.findOne({ _id: req.params.id, userId: currentUser._id });

    if (!noteToDelete) {
      return res.status(404).json({ error: 'Note not found or unauthorized.' });
    }

    await Note.deleteOne({ _id: req.params.id });
    res.status(200).json({ message: 'Note successfully destroyed.' });
  } catch (err) {
    res.status(500).json({ error: 'Failed to delete note.' });
  }
});

```

Step 5.4: Test Your Live API via Frontend Interface

To see this API work inside a browser, replace the code inside your protected `/dashboard` route block in `app.js` with this dynamic HTML/JavaScript interface:

```

app.get('/dashboard', (req, res) => {
  if (req.session.user) {
    res.send(`
      <h1>Dashboard for ${req.session.user}</h1>
      <a href="/logout">Logout</a>
      <hr>
    `);
  }
});

```

```

<h3>Create a New Private Note</h3>
<input type="text" id="title" placeholder="Note Title"><br><br>
<textarea id="content" placeholder="Write something..."></textarea><br><br>
<button onclick="saveNote()">Save Note</button>

<h3>Your Saved Notes</h3>
<div id="notesContainer">Loading notes...</div>

<script>
  // Fetch and display notes as soon as the dashboard loads
  async function loadNotes() {
    const response = await fetch('/api/notes');
    const notes = await response.json();
    const container = document.getElementById('notesContainer');
    container.innerHTML = '';

    if(notes.length === 0) {
      container.innerHTML = '<p>No notes found. Create your first one above!</p>';
      return;
    }

    notes.forEach(note => {
      container.innerHTML += `
        <div style="border: 1px solid #ccc; padding: 10px; margin: 10px 0; border-radius:5px;">
          <h4>\${note.title}</h4>
          <p>\${note.content}</p>
          <button onclick="deleteNote('\${note._id}')" style="color:red;">Delete</button>
        </div>
      `;
    });
  }

  // Send data via JSON to our REST API
  async function saveNote() {
    const title = document.getElementById('title').value;
    const content = document.getElementById('content').value;

    await fetch('/api/notes', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ title, content })
    });

    document.getElementById('title').value = '';
    document.getElementById('content').value = '';
    loadNotes();
  }

  // Trigger a secure DELETE command
  async function deleteNote(id) {
    await fetch('/api/notes/' + id, { method: 'DELETE' });
    loadNotes();
  }

  loadNotes();
</script>
`);
} else {
  res.status(401).send('Unauthorized. Please <a href="/login.html">login</a>.');
}
});

```

Step 5.5: Run the Server

1. Fire up your script locally using `nodemon app.js`.
2. Open your browser, register/login, and navigate to your dashboard.
3. Type in a few sample notes and save them. They will immediately save to your MongoDB cloud database cluster and render onto your screen without reloading the page.
4. If you log out and create a completely different user account, that dashboard will be empty because your MongoDB queries filter data specifically by the active user's structural ID mapping.